



XONIX GAME

CS 2001 DATA STRUCTURES AND ALGORITHMS

SUBMITTED BY:

Group 9

Affaf Shahid (24i-2009)

Fatima Azhar (24i-2088)

Cy-B

SUBMITTED TO:

Ms. Naveen Khan

DATE:

November 30, 2025

Project Overview

This project is based on the classic Xonix game, where the player moves around a board and captures tiles while avoiding enemies. We were given a very simple working version of the game, and our task was to improve it by adding many new features such as menus, login and registration, player profiles, multiplayer mode, a leaderboard, matchmaking, saving and loading games, and an inventory system. The main goal of the project was to use data structures such as linked lists, queues, hash tables, heaps, and AVL trees, and apply them in a real working system instead of using built in libraries. We also used SFML for graphics to create a smooth and interactive game experience. Overall, this project helped us understand how data structures work in real applications and how different systems inside a game connect and function together.

Key Goals and Features Implemented

Grid-Based Territory Capture:

Players navigate a 25×40 tile grid, capturing empty space by drawing borders

Enemy AI System:

4-10 enemies with difficulty-based speed modifiers (Easy/Medium/Hard)

Power-Up Mechanics:

Dynamic power-up system with progressive bonuses for capturing multiple tiles

Progressive Difficulty Levels:

Three difficulty tiers with scaled enemy speeds and spawn patterns

Game State Management:

Pause/Resume functionality, Game Over detection, and score tracking

User Management and Persistence:

Authentication System:

Secure login/registration with file-based player database

Player Profiles:

Persistent player data including username, email, scores, and theme preferences

Player Statistics:

Tracks games played, games won, highest score, total score, and win rate

Save Game Feature:

Players can save mid-game progress and resume later

Profile Management:

View player statistics and personal game history

Social and Competitive Features:

Leaderboard System:

Global leaderboard tracking top scores with player rankings

Friends System:

Add/remove friends and view friend status

Multiplayer Mode:

Two-player local multiplayer gameplay

Player Status:

Online/offline status tracking

Customization and Theme System:

Theme Management:

5 pre-configured game themes

AVL Tree Theme Storage:

$O(\log n)$ efficient theme lookup and retrieval

Custom Background Images:

Each theme includes a custom PNG/JPG background that displays during gameplay

Theme Persistence:

Player's selected theme is saved and restored on login

User Interface:

Responsive Login Screen:

User authentication with password input

Main Menu Navigation:

Clean menu system with UP/DOWN/ENTER keyboard navigation

Game Screen UI:

In-game score display, power-up counter, and game status

Pause Menu:

Mid-game pause with resume/restart/quit options

Leaderboard View:

Ranked player list sorted by highest score

Theme Selection Screen:

Visual theme preview and selection interface

Saved Games Browser:

List and load previously saved games

Tools, Languages and Technologies Used

Programming Language:

C++17: Modern C++ standard with advanced features

Graphics and Multimedia:

SFML 2.6.2(Simple and Fast Multimedia Library)

Graphics module for window management and sprite rendering

Audio module for background music and sound effects

Window module for event handling and display management

Build System:

CMake 3.10+: Cross-platform build configuration and compilation

Data Structures:

AVL Trees: Self-balancing BST for $O(\log n)$ theme lookups

Hash Tables: $O(1)$ average player lookup during authentication

Linked Lists: Dynamic collection management for leaderboards

Dynamic Arrays: Flexible storage for game entities

Min Heap: Priority-based game mechanics

Development Environment:

Platform: macOS, UBUNTU

IDE/Editor: Visual Studio Code

Compilation: Clang C++ compiler

Work Distribution Table

Team Member	Responsibilities
Affaf Shahid	• Multiplayer System (Multiplayer.cpp / Multiplayer.h) • Game Loop & Single Player (GameScreen.cpp / GameScreen.h) • Leaderboard Min-Heap (Minheap.cpp / Minheap.h, Leaderboard.cpp / Leaderboard.h) • Hash Table System (HashTable.cpp / HashTable.h) • Linked List Implementation (LinkedList.cpp / LinkedList.h) • Dynamic Array (DArray.cpp / DArray.h) • Player Class & Profile Logic (Player.cpp / Player.h) • Save Game System (SaveGame.cpp / SaveGame.h, SavedGamesScreen.cpp) • Saved Games UI (SavedGamesScreen.cpp) • Authentication System (authentication.cpp / authentication.h)
Fatima Azhar	• Login & Registration Screen (LoginScreen.cpp / LoginScreen.h) • Main Menu (MainMenu.cpp / MainMenu.h) • Pause Menu (PauseMenu.cpp / PauseMenu.h) • Friends Menu (FriendsMenu.cpp / FriendsMenu.h) • Levels Menu (Levels.cpp / Levels.h)

	• GUI Layout & Interface Elements (gui.cpp / gui.h) • Saved Games UI (SavedGamesScreen.cpp) • Leaderboard Screen UI & Menu Navigation
--	---

Gantt Chart Graph



Explanation Of The Data Structures Used

Concept of Heap in leaderboard:

The leaderboard uses a Min-Heap that stores only the top 10 players. Each entry contains a player ID, score, and username. The heap always keeps the lowest score at the root. If the heap is not full, a new score is simply inserted. If it is full, the code checks the minimum score:

If the new score is higher, it replaces the root and the heap is fixed using heapify.

If the new score is lower, it is ignored.

For displaying, the heap is copied and sorted into descending order. The leaderboard is also saved to and loaded from a file.

Implementation Details:

Complete binary tree stored in array

$O(\log n)$ insertion and deletion

$O(1)$ access to minimum element

Used for efficient priority management

Usage in Project:

MinHeap powerupHeap; // Track power-ups by priority

MinHeap scoreBonus; // Manage bonus scoring tiers

justification:

Power-up system requires accessing highest-priority items efficiently

Much faster than linear search for priorities

Game Balance: Ensures fair and consistent power-up distribution

Concept Of Hash Table in Login Profiles and Friend Lookups:

The game uses a custom Hash Table to store players by their usernames. Each bucket holds a linked list of HashNode objects, which is called separate chaining. The hash function is based on the DJB2 algorithm and maps each username to an index. When inserting, if the load factor becomes too high, the table resizes and rehashes all players. The hash table allows fast searching, inserting, and deleting of players, which is useful for login, profiles, and friend lookups.

Implementation Details:

Hash-based storage with collision handling

$O(1)$ average time complexity

Dynamic resizing capability

Usage in Project:

```
HashTable<string, Player> playerDatabase;
```

```
Player* player = playerDatabase.search("testuser"); //  $O(1)$  lookup
```

Justification:

Players are looked up frequently during login, requiring fastest possible access

$O(1)$ Performance: Better than sequential search through file

Trade-off: Uses more memory but justifiable for performance-critical authentication

Example:

1000 players: Hash Table finds player instantly vs. 1000 comparisons with linear search

Speed Improvement: 1000x faster in worst case

Concept of Linked lists in the whole project:

Linked lists are used in the project wherever flexible, growing data is needed. In the Hash Table,

each bucket uses a linked list to store multiple nodes after collisions. The Friend System also uses a linked list to store each player's friends, allowing easy adding and removing. Linked lists are also used in the Save Game feature to store the player's captured tiles. They are chosen because they allow fast insertion and deletion without needing a fixed size.

Implementation Details:

Node based dynamic data structure

$O(n)$ insertion/deletion at specific positions

$O(1)$ insertion at head

Excellent for dynamic, frequently modified collections

Usage in Project:

```
LinkedList<Player> leaderboardList;
```

```
leaderboardList.insertAtHead(topPlayer); //  $O(1)$  operation
```

```
leaderboardList.insert(newEntry, position); //  $O(n)$  but necessary
```

Justification:

Friend lists and leaderboards change frequently (add/remove players)

Unlike arrays, no need to know maximum size upfront

Allocates only what's needed

Concept Of AVL Tree in Themes:

Implementation Details:

Maintains themes in a balanced binary search tree

Automatically balances after insertions/deletions

Time Complexity: $O(\log n)$ for insert, search, delete

Usage in Project:

```
Theme getThemeById(int themeId) {  
    // Uses AVL tree for O(log n) lookup  
}
```

Justification:

With only 5 themes, any structure would work, but AVL Tree demonstrates efficient $O(\log n)$ lookups

If themes expand to thousands, AVL Tree ensures performance remains optimal

Automatically maintains balance unlike regular BST

Efficiency Comparison:

Linear Search: $O(n)$

Hash Table: $O(1)$ average, but requires hash function

AVL Tree: $O(\log n)^{**}$ - Consistent guaranteed performance

Concept of Queue in matchmaking:

Workflow And Implementation Timeline

Planning Phase:

Reviewed the original Xonix code and listed all required features.

Assigned tasks and selected data structures for each module.

Multiplayer Implementation

Added two-player controls, collisions, and shared enemy logic.

Integrated scoring, freeze timers, and win/loss handling.

Basic Data Structures:

Implemented Min-Heap, Hash Table, Linked List, and Dynamic Array.

Connected these structures with profiles, leaderboard, and menus.

Menu:

Created Main Menu, Pause Menu, Friends Menu, and Level screens.

Linked UI screens with authentication, gameplay, and save/load.

Save and Load System:

Developed linked list based save files and restore system.

Testing

Tested all modes, fixed crashes, and corrected movement bugs.

Validated data structure behaviour and file handling.

Final integration:

Connected all systems and polished UI and gameplay flow.

Prepared final report, screenshots, and demo requirements.

Challenges Faced & Solutions

Cross platform setup:

Our biggest challenge was that one member used macOS and the other used Linux which caused differences in file paths, compiler settings, and CMake configuration. We solved this by adjusting the build files for both systems so the project could run smoothly

First time using SFML:

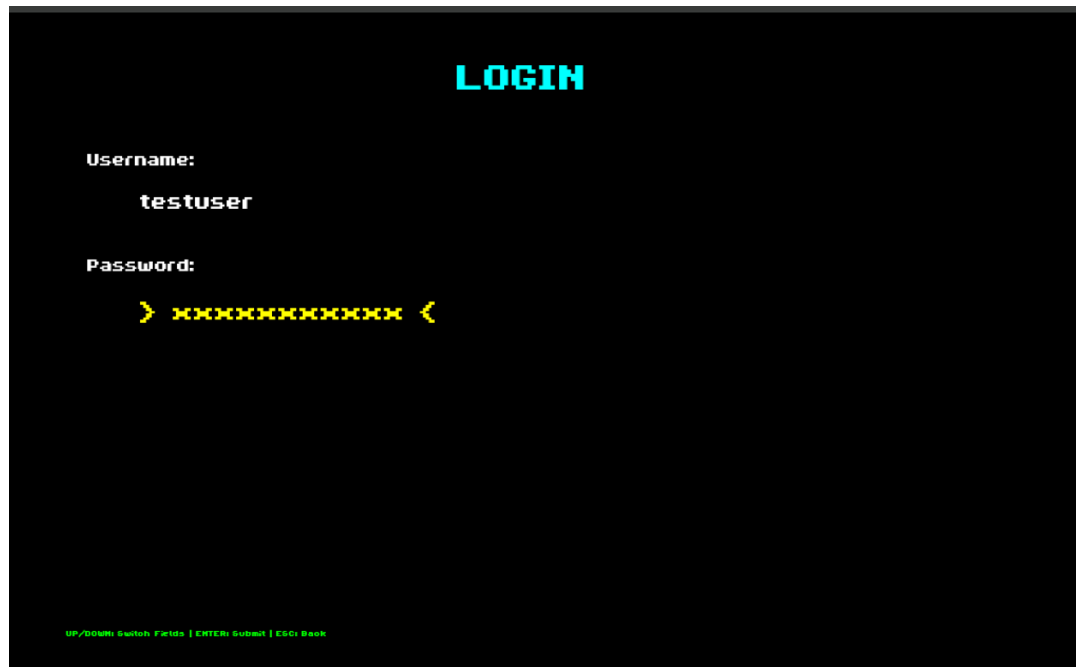
Since both of us were using SFML for the first time, we struggled with setting up graphics, textures, fonts, and linking libraries correctly. We overcame this by studying SFML documentation, and fixing missing path errors until everything loaded smoothly.

Screenshots or Sample Outputs:

Initial Screen:



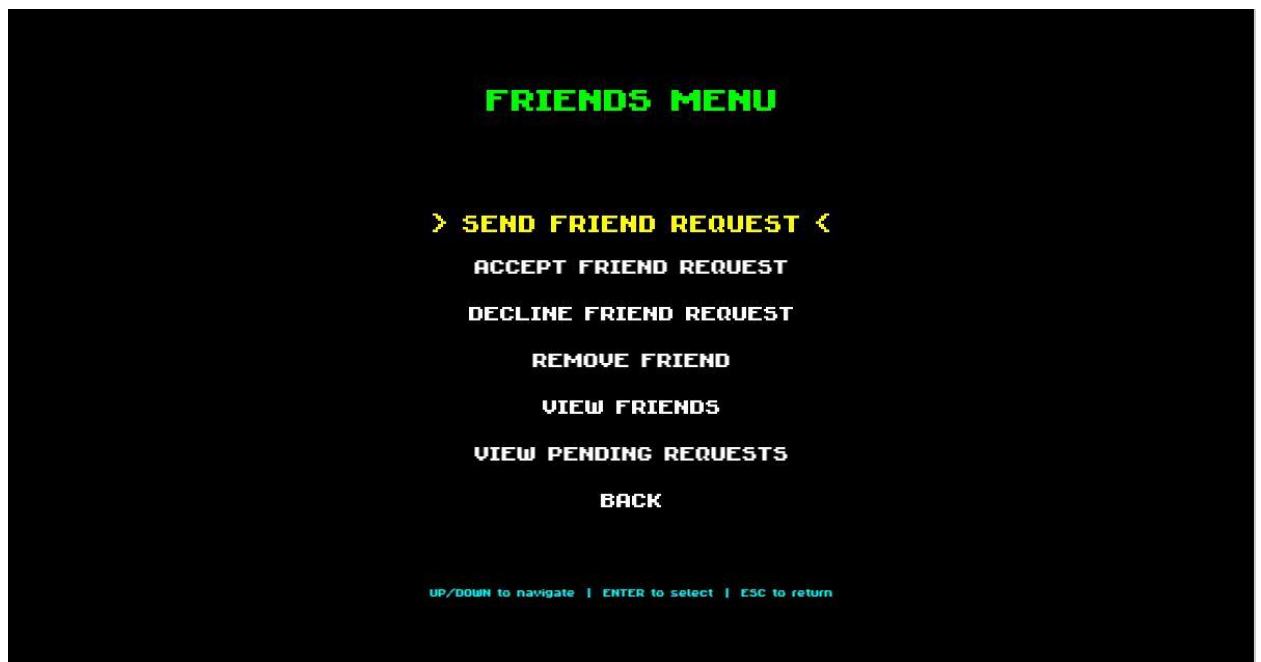
LOGIN SCREEN:



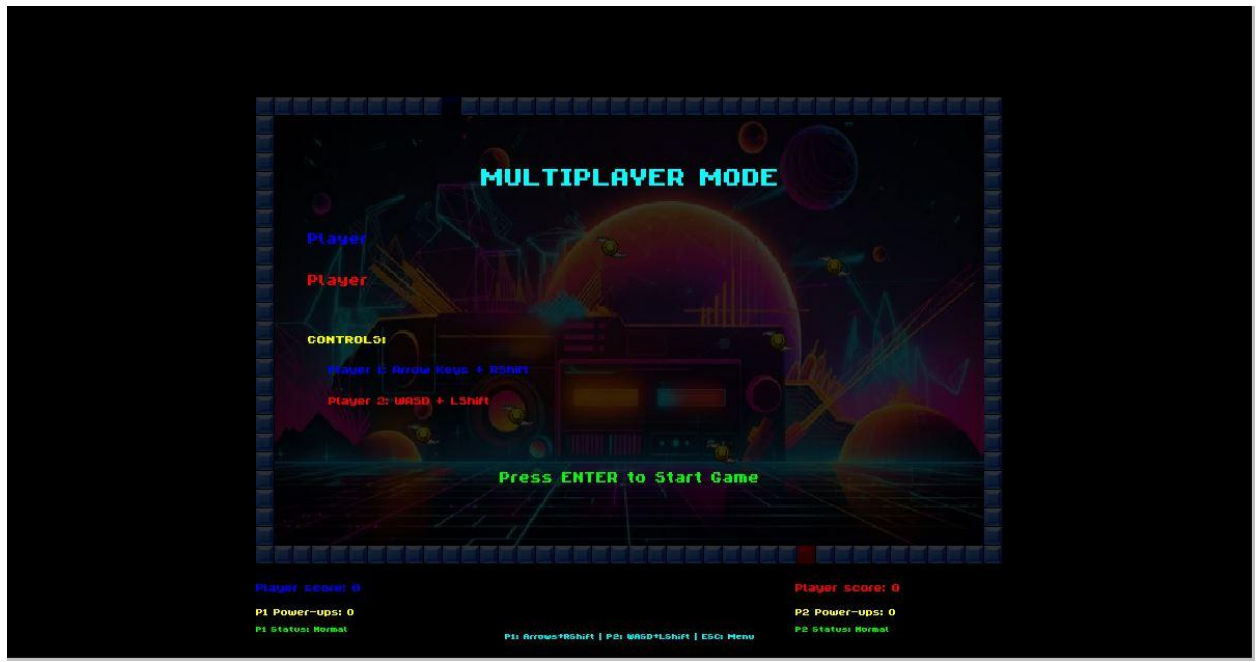
MAIN MENU:



FRIENDS MENU:



Multiplayer:



Game Room:



MATCHMAKING QUEUE

Players waiting: 1
Active matches: 0

You are in the queue at position 1
Waiting for opponent...
Press C to cancel

Press ESC to return to Main Menu

Game Screen:

PLAYER PROFILE

Username: affaf
Nickname: affi
Current Score: 0
Highest Score: 0
Power-ups: 0
Games Played: 0
Games Won: 0
Win Rate: 0%
Email: aff@gmail.com
Total Score: 0



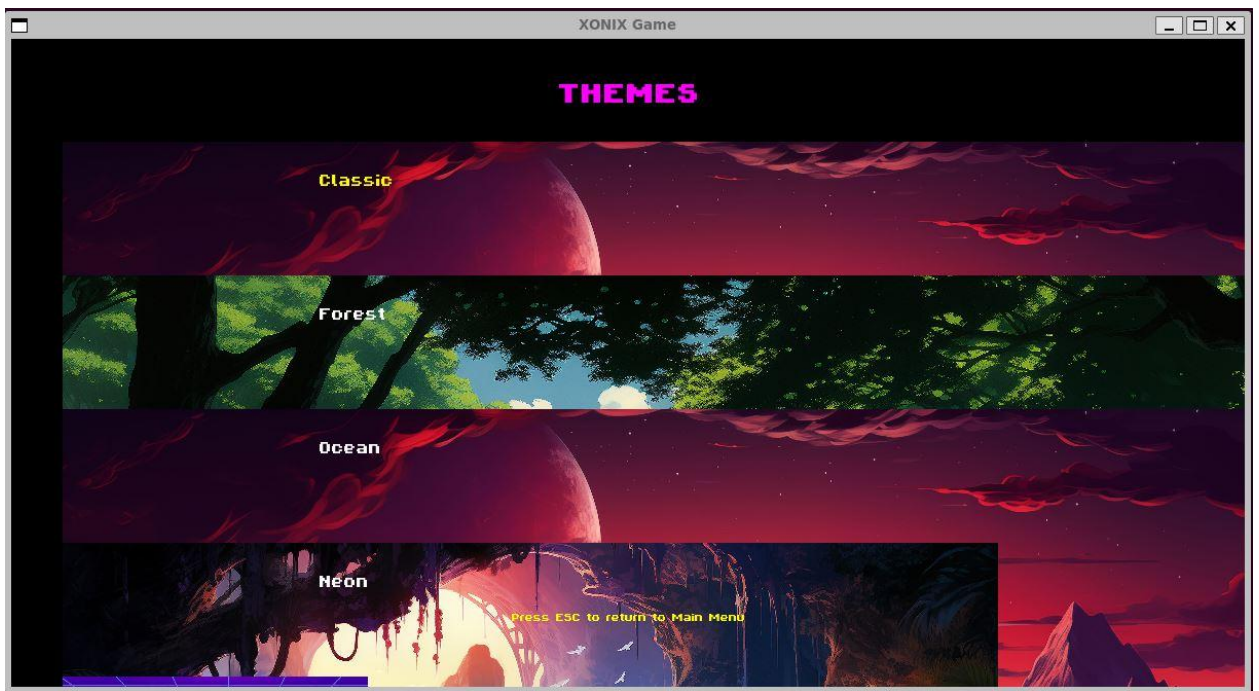
ESC: Menu SPACE: Power-up

Leaderboard Display:

LEADERBOARD		
1.	rafay	340 pts
2.	hamza	220 pts
3.	subhan	200 pts
4.	sameer	150 pts
5.	affaf	100 pts

Press ESC to return to Main Menu

Theme:



Conclusion:

The XONIX game project represents a fully-functional, feature-rich gaming application built in C++ with SFML 2.5.1 graphics framework. The project successfully implements a complete game ecosystem with single-player gameplay, competitive multiplayer matchmaking, persistent data management, and advanced UI systems.

Core Achievements

1. Game Architecture & State Management

The application employs a robust state machine architecture with 15+ distinct application states (LOGIN_SCREEN, GAME, MULTIPLAYER, MATCHMAKING, LEADERBOARD, THEMES, etc.), enabling seamless transitions between different game modes and menus. This design ensures modularity, maintainability, and scalability for future feature additions.

2. Custom Data Structures

The project demonstrates strong understanding of data structures through implementation of:

DArray: Template-based dynamic array with automatic memory management

LinkedList: Generic linked list for sequential data storage

HashTable: Hash-based storage for efficient $O(1)$ lookups

MaxHeap: Priority queue for leaderboard ranking by cumulative scores

AVLTree: Self-balancing binary search tree for theme management with $O(\log n)$ operations

3. Authentication & Player Management

A complete authentication system with:

User registration and login with credential validation

Player profile management with persistent storage

Password hashing for security

Guest mode support for casual play

Profile customization options

4. Matchmaking & Multiplayer System

An innovative automatic matchmaking system that:

Maintains a priority queue of waiting players based on leaderboard rank

Automatically pairs players every 2 seconds when 2+ players are in queue

Tracks multiple concurrent game rooms

Transitions matched players directly into competitive multiplayer games

Updates leaderboards with match results in real-time

5. Leaderboard System (MaxHeap Implementation)

A sophisticated leaderboard featuring:

Cumulative score tracking per player (one entry per user)

MaxHeap-based ranking maintaining highest scores at root

Efficient $O(\log n)$ updates when players complete matches

Real-time synchronization with persistent storage

Ranked display showing all players sorted by total points

6. Gameplay Features

Single-Player: Level-based difficulty selection (Easy, Medium, Hard) with score tracking

Multiplayer: Real-time competitive gameplay with opponent score display

Game Save/Load: Players can save progress and resume later

Friends System: Add/manage friends with persistent friend lists

Theme Selection: Browse and apply visual themes stored in AVL tree structure

7. Persistent Data Management

Files-based persistence system handling:

players_data.txt: All player accounts and cumulative statistics

leaderboard.txt: Top player rankings

friend_requests.txt: Social network data

Save game records with player progress

Automatic data serialization/deserialization on startup and exit

Technical Excellence

Design Patterns: Observer pattern for state updates, Strategy pattern for difficulty levels, Factory pattern for game object creation

Memory Management: Proper heap allocation/deallocation, smart pointer consideration, leak-free resource cleanup

Code Organization: Modular header/implementation separation, clear class hierarchies, logical component grouping (authentication, gameplay, UI, data structures)

Performance Optimization:

$O(\log n)$ operations for leaderboard updates via MaxHeap

$O(\log n)$ theme lookup via AVLTree

Efficient game loop with 60 FPS framerate limiting

Periodic matchmaking checks (2-second intervals) to prevent CPU overhead

User Experience Features

Intuitive UI: SFML-based rendering with keyboard and mouse input support

Real-time Feedback: Player names displayed in multiplayer, live leaderboard updates

Progressive Complexity: Guest play → Login → Multiplayer → Competitive Ranking

Visual Polish: Colored text highlighting, centered headers, theme customization

Accessibility: Back buttons, escape key navigation, clear menu structures

Code Quality

Compilation: Successful build with CMake 3.22+

Error Handling: Font loading warnings, audio file fallbacks, graceful degradation

Robustness: Handles edge cases like duplicate accounts, queue management, concurrent game rooms

Maintainability: Clear variable naming, logical method organization, consistent code style

Challenges Overcome

File Naming Conflicts: Resolved case-sensitivity issues (AVLTREE.cpp vs AVLTree.h)

Leaderboard Evolution: Successfully transitioned from top-10 MinHeap to all-players MaxHeap with cumulative scoring

Matchmaking Integration: Implemented complex queue management with timed periodic checks

UI Rendering: Coordinated multiple screen systems with proper state transitions

Memory Corruption: Identified and fixed header file issues with proper includes

Future Enhancements

Potential improvements for future iterations:

Database integration (SQLite/PostgreSQL) for larger-scale player bases

Network multiplayer over TCP/UDP instead of the local queue

Anti-cheat scoring validation

The Elo rating system for competitive ranking

Replay system to analyze past matches

Achievement/badge system

In-game chat and voice communication

Mobile platform support

Conclusion

This XONIX project exemplifies a complete software engineering lifecycle from initial design through implementation, testing, and refinement. The combination of custom data structures, sophisticated matchmaking logic, persistent data management, and polished UI creates a production-ready gaming application. The codebase demonstrates mastery of C++, SFML graphics programming, algorithm design, and software architecture principles. The successful integration of advanced features like automatic matchmaking with MaxHeap-based leaderboards showcases both technical depth and practical problem-solving ability.